

Buổi 12 — Khử nhiễu và miền tần số

1. Mục tiêu

Sau buổi này bạn:

- Phân biệt **bộ lọc nhân quả** (chỉ dùng quá khứ) với bộ lọc nhìn tương lai, bằng trung bình trượt 3 điểm tính tay hai kiểu.
- Đọc **phổ** của một chuỗi: nhịp nào mạnh, nhiễu nằm ở tần số nào, chuỗi có đáng khử nhiễu không.
- Giải thích **Nyquist** và **aliasing** bằng hình ảnh bánh xe quay ngược trong phim, rồi chỉ ra bằng số vì sao hạ mẫu mà không lọc trước tạo ra chu kỳ giả.
- Chọn bộ lọc trong 6 họ (trung bình trượt, EWMA, Savitzky-Golay, Butterworth, Kalman, wavelet) theo **khi nào dùng, khi nào không**, và đo **trễ** của từng cái.
- Viết **bài kiểm nhân quả tự động**: đổi phần cuối chuỗi, xem đầu ra ở phần trước có đổi không.
- Đo **cái giá của rò rỉ**: feature nhìn tương lai và mục tiêu đã làm trơn cho sai số đẹp giả tạo bao nhiêu.

2. Nhắc lại buổi trước

Từ buổi 4-9:

- **Trung bình trượt**: `rolling(k).mean()` lấy k điểm **kết thúc** tại điểm hiện tại; `center=True` lấy k điểm **quanh** nó, nửa cửa sổ nằm ở tương lai.
- **Tương quan chéo** (buổi 8): dịch một chuỗi k bước rồi tính tương quan; k cho tương quan lớn nhất là độ trễ giữa hai chuỗi.
- **Tần số** = $1 / \text{chu kỳ}$: lặp mỗi 24 giờ là tần số $1/24$ chu kỳ mỗi giờ. **Phổ** (buổi 9, hộp Mượn trước) cho biết mỗi tần số góp bao nhiêu phần dao động của chuỗi.

Từ buổi 10-11:

- **Nhân quả**: kết quả tại t chỉ dùng dữ liệu tới t . Cách diễn và bộ lọc Hampel `center=True` nhìn tương lai: làm sạch lịch sử thì được, làm feature dự báo thì là rò rỉ.
- **Kiểm rò rỉ**: chạy một hàm trên dữ liệu đầy đủ và trên dữ liệu bị cắt, so phần trước mốc cắt.
- **Dịch mức** là tín hiệu, không phải nhiễu: khử nhiễu không được xóa nó.

3. Trạng thái đầu buổi

Sau `python lab.py up` (trong thư mục `lab/`):

Hạng mục	Trạng thái
Dữ liệu	<code>du-lieu/raw/uci-appliances-energy/energydata_complete.csv</code> — 19.735 dòng, mỗi 10 phút một mẫu, 11/1/2016 → 27/5/2016, sha256 <code>2fccf3544458</code>
Cột dùng	<code>Appliances</code> : điện thiết bị trong một ngôi nhà (Wh mỗi 10 phút), rất nhiễu; <code>T2</code> : nhiệt độ phòng khách (°C), rất trơn
Nguồn	UCI Appliances Energy Prediction (Candanedo, 2017), CC BY 4.0
Môi trường	Python 3.12; numpy 2.5.3, pandas 3.0.5, scipy 1.18.1, statsmodels 0.15.0, PyWavelets 1.8.0

Hạng mục	Trạng thái
code/ khu_nhieu.py	doc_cam_bien, pho, cong_suat_quanh, ha_mau, 10 bộ lọc (bo_loc_mac_dinh), kiem_nhan_qua, tre_pha, bang_bo_loc, danh_gia_feature, gia_cua_ro_ri, cham_tren_muc_tieu_lam_tron
code/lab.ipynb	notebook của Lab, bước 1-5
Đang cố tình sai	kiem_nhan_qua bỏ qua 200 điểm cuối trước điểm đổi; ha_mau bỏ qua cờ loc_truoc; danh_gia_feature mặc định dùng trung bình trượt có tâm; cham_tren_muc_tieu_lam_tron chỉ trả sai số trên mục tiêu đã làm trơn
Triệu chứng	bảng bộ lọc báo trung bình trượt có tâm, Savitzky-Golay, Kalman smoother là "nhân quả"; phổ sau hạ mẫu có đỉnh 2,5 giờ không ai giải thích được; báo cáo khoe MAE 35,8 thay vì 46,8
python lab.py check lúc này	ĐỎ: 6/8 test hỏng

4. Lý thuyết

Từ mới trong buổi

Thuật ngữ	Nghĩa một câu	Ví dụ
feature / mục tiêu	Feature: cột đầu vào mô hình dùng để dự báo. Mục tiêu: con số cần dự báo, cũng là con số dùng để chấm.	Feature: điện trung bình 2 giờ qua; mục tiêu: điện giờ tới.
tín hiệu / nhiễu	Tín hiệu: phần có quy luật ta muốn giữ. Nhiễu: phần ngẫu nhiên không dự báo được.	Nhịp ngày là tín hiệu; bật tắt ấm đun là nhiễu.
bộ lọc (filter)	Phép biến đổi chuỗi thành chuỗi khác, thường để làm trơn.	Trung bình trượt 3 điểm.
bộ lọc nhân quả	Đầu ra tại t chỉ dùng dữ liệu tới t .	Trung bình 3 điểm gần nhất.
trễ (pha)	Đầu ra bộ lọc chạy sau tín hiệu thật bao nhiêu bước.	Trung bình 13 điểm gần nhất trễ 6 bước.
tần số lấy mẫu f_s	Số mẫu mỗi đơn vị thời gian.	10 phút một mẫu: $f_s = 6$ mẫu/giờ.
tần số Nyquist	Tần số cao nhất còn thấy đúng: $f_s/2$.	$f_s = 6$ mẫu/giờ $\rightarrow$ Nyquist 3 chu kỳ/giờ.
aliasing	Dao động nhanh hơn Nyquist giả dạng thành dao động chậm.	Bánh xe trong phim quay ngược.
hạ mẫu (downsampling)	Giảm tần số lấy mẫu, ví dụ từ 10 phút sang 1 giờ.	Lấy mỗi 6 mẫu một mẫu.
lọc thông thấp	Giữ dao động chậm (tần số thấp), bỏ dao động nhanh.	Lọc trước khi hạ mẫu.
periodogram, Welch	Periodogram: phổ tính một lần trên cả chuỗi (lờm chờm). Welch: chia chuỗi thành nhiều đoạn chồng nhau, lấy trung bình phổ các đoạn (mượt hơn).	Mục 4.2.
EWMA	Trung bình trượt hàm mũ: $z_t = \alpha y_t + (1 - \alpha)z_{t-1}$.	Mục 4.4.
Savitzky-Golay	Khớp một đa thức bậc thấp trên cửa sổ quanh mỗi điểm, lấy giá trị đa thức tại điểm đó.	Mục 4.4.

Thuật ngữ	Nghĩa một câu	Ví dụ
Butterworth	Bộ lọc cắt tần số theo một ngưỡng; tính mỗi đầu ra từ cả đầu vào lẫn các đầu ra trước.	Mục 4.4.
<code>sosfilt</code> / <code>sosfiltfilt</code>	Chạy bộ lọc một chiều theo thời gian / chạy xuôi rồi chạy ngược lại (bản ổn định của <code>lfilter</code> / <code>filtfilt</code>).	Mục 4.4.
Kalman filter / smoother	Ước lượng tín hiệu ẩn từ một mô hình; filter chỉ dùng quá khứ, smoother dùng cả chuỗi.	Mục 4.4.
wavelet	Tách chuỗi thành nhiều thang (nhanch, chậm), bỏ hệ số nhỏ rồi ghép lại.	Hộp Nâng cao mục 4.4.
RMSE	Căn của trung bình bình phương sai số.	Sai số 1, $-3 \rightarrow \text{căn}(5) \approx 2,24$.

4.1 Nhiễu, bộ lọc, và câu hỏi "có nhìn tương lai không"

Vấn đề. Chuỗi = tín hiệu + nhiễu. Khử nhiễu là ước lượng phần tín hiệu. Nhưng nhiễu bộ lọc cho đường đẹp nhất lại dùng dữ liệu **sau** thời điểm đang tính. Mô tả lịch sử thì không sao; làm feature dự báo thì chấm trên quá khứ (backtest) đẹp mà dùng thật thì sập.

Ví dụ số nhỏ — tự tính tay. Năm giờ điện: (10, 12, 14, 30, 16). Trung bình trượt 3 điểm, hai kiểu:

Giờ	1	2	3	4	5
Điện	10	12	14	30	16
Kiểu trailing (3 giờ gần nhất)	—	—	12	18,67	20
Kiểu centered (giờ trước, giờ này, giờ sau)	—	12	18,67	20	—

Độc bảng. Giờ 3 kiểu trailing là $(10 + 12 + 14) / 3 = 12$; kiểu centered là $(12 + 14 + 30) / 3 = 18,67$, đã chứa số 30 của giờ 4. Đứng ở giờ 3, chưa ai biết giờ 4 là 30: kiểu centered đang nhìn tương lai.

Thử đổi giờ cuối từ 16 thành 40. Kiểu centered ở giờ 4 đổi từ 20 thành $(14 + 30 + 40) / 3 = 28$: một con số **quá khứ** bị sửa khi dữ liệu mới về. Kiểu trailing ở giờ 4 vẫn **18,67**. Đó là toàn bộ ý tưởng của bài kiểm nhân quả (mục 4.5).

Khi nào dùng, khi nào không. Kiểu centered (và mọi bộ lọc nhìn tương lai) dùng để **mô tả** lịch sử, tách xu hướng (buổi 6), vẽ báo cáo. Không bao giờ dùng làm feature dự báo hay làm mục tiêu để chấm. Kiểu trailing dùng được cho dự báo, đổi lại nó **trễ**: giờ 4 tăng vọt lên 30 mà đầu ra mới lên 18,67.

Tóm lại. Bộ lọc nhân quả chỉ dùng quá khứ nên trễ; bộ lọc nhìn tương lai không trễ nhưng sửa cả quá khứ khi dữ liệu mới về. Câu hỏi đầu tiên với mọi bộ lọc: đầu ra tại t có dùng $y_{t+1}, y_{t+2}, \dots$ không?

Tự kiểm tra. Chuỗi (4, 8, 6, 10). Tính trung bình trượt 3 điểm kiểu trailing và centered tại điểm thứ ba. Kiểu nào dùng số 10?

Đáp án (tự làm trước rồi mới đọc)

Trailing: $(4 + 8 + 6) / 3 = 6$. Centered: $(8 + 6 + 10) / 3 = 8$, có dùng số 10 của điểm thứ tư, tức nhìn tương lai. Nhầm hay gặp: nghĩ `rolling(3).mean()` là centered; mặc định của pandas là trailing, phải ghi `center=True` mới thành centered.

4.2 Đọc phổ: nhịp nào mạnh, nhiều nằm ở đâu

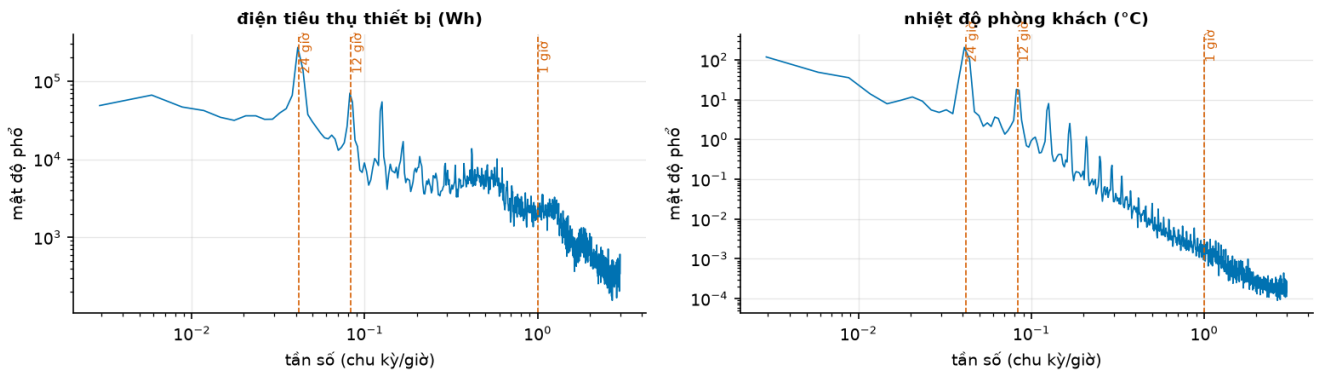
Vấn đề. Trước khi lọc, cần biết chuỗi có gì để lọc: dao động nhanh chiếm bao nhiêu, và nhịp nào phải giữ.

Trực giác. Mọi chuỗi đều viết được thành tổng nhiều sóng đều đặn. Phổ là bảng "mỗi sóng to cỡ nào". Nhiều kiểu bật tắt thiết bị là dao động rất nhanh, nằm ở phía tần số cao; nhịp sinh hoạt ngày nằm ở tần số 1/24.

Ví dụ số nhỏ — tự tính tay. Chuỗi (1, -1, 1, -1, 1, -1) lên xuống mỗi bước: lặp mỗi 2 bước, tần số 1/2, đúng bằng tần số cao nhất còn thấy được (mục 4.3). Toàn bộ dao động nằm ở tần số đó. Chuỗi (1, 1, 1, -1, -1, -1) lặp mỗi 6 bước: năng lượng dồn ở tần số 1/6, thấp hơn.

```
f, P = signal.welch(y - y.mean(), fs=6.0, nperseg=2048) # fs = 6 mẫu/giờ; f tính bằng chu kỳ/giờ
chu_ky_manh_nhat = 1 / f[np.argmax(P)] # giờ
```

Phổ công suất: cả hai cảm biến có nhịp ngày rõ, nhưng điện tiêu thụ còn nhiều năng lượng ở tần số cao



Cách đọc hình.

1. **Trục ngang** (cả hai ô): tần số, chu kỳ mỗi giờ, thang log; bên trái là dao động chậm, bên phải là dao động nhanh.
2. **Trục dọc:** mật độ phổ (độ lớn của mỗi tần số), thang log.
3. **Ký hiệu:** đường xanh là phổ Welch; vạch cam đứt đánh dấu chu kỳ 24 giờ, 12 giờ, 1 giờ.
4. **Nhìn vào đâu:** đỉnh ở vạch 24 giờ; phần bên phải vạch 1 giờ.
5. **Kết luận:** cả hai có đỉnh ở chu kỳ khoảng một ngày (nhịp sinh hoạt); điện thiết bị còn 17,8% năng lượng ở dao động nhanh hơn một giờ, nhiệt độ gần như không có.

Nhiệt độ phòng không thể nhảy trong vài phút, nên lọc T2 chỉ thêm trễ; Appliances mới có chỗ để khử nhiễu.

Khi nào dùng, khi nào không. Đọc phổ trước mọi quyết định lọc và hạ mẫu. Đừng đọc phổ của chuỗi còn xu hướng mạnh mà chưa trừ đi: xu hướng dồn năng lượng vào tần số thấp, che các đỉnh khác.

Tóm lại. Phổ cho biết dao động của chuỗi nằm ở tần số nào. Năng lượng ở tần số cao gần 0 thì lọc chỉ thêm trễ.

Tự kiểm tra. Dữ liệu theo giờ có đỉnh phổ ở tần số 1/168 chu kỳ mỗi giờ. Đó là nhịp gì?

Đáp án (tự làm trước rồi mới đọc)

Chu kỳ = $1 / (1/168) = 168$ giờ = **một tuần**: nhịp tuần. Nhầm hay gặp: đọc 1/168 là "168 lần mỗi giờ"; tần số là số vòng mỗi giờ, nên 1/168 nghĩa là một vòng mỗi 168 giờ.

4.3 Nyquist và aliasing: bánh xe quay ngược trong phim

Vấn đề. Hạ mẫu (10 phút → 1 giờ) là việc làm hằng ngày. Làm sai, nó **tạo ra** một chu kỳ không có thật, và ta đi tìm lời giải thích kinh doanh cho một thứ không tồn tại.

Trực giác. Trong phim cũ, bánh xe ngựa chạy nhanh trông như quay chậm, thậm chí quay ngược. Máy quay chụp 24 hình mỗi giây. Nếu giữa hai hình, bánh xe quay gần trọn một vòng, thiếu một chút, thì mắt thấy nó lùi lại một chút mỗi hình: quay ngược, chậm. Chuyển động nhanh bị lấy mẫu thưa đã **giả dạng** thành chuyển động chậm. Muốn thấy đúng, mỗi vòng quay phải được chụp ít nhất hai lần: tần số lấy mẫu phải gấp đôi tần số của chuyển động.

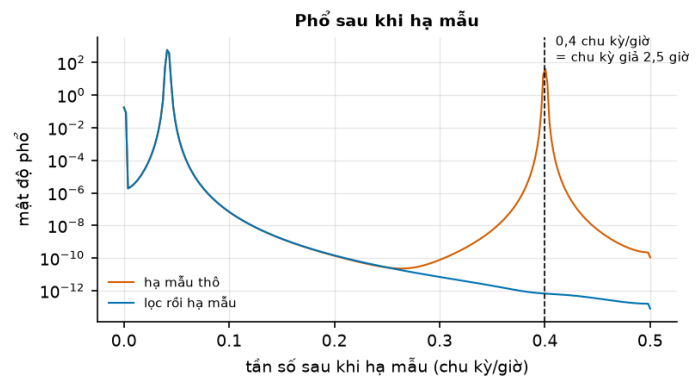
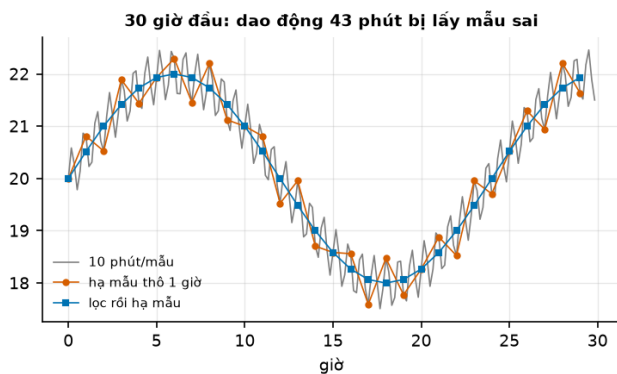
Ví dụ số nhỏ — tự tính tay. Sóng lặp mỗi 4 bước: (0, 1, 0, -1, 0, 1, 0, -1, ...). Lấy mỗi 3 bước một mẫu (vị trí 0, 3, 6, 9, 12, 15): ra (0, -1, 0, 1, 0, -1). Chuỗi mẫu lặp mỗi 4 mẫu, tức mỗi 12 bước gốc: một sóng **chậm gấp ba** sóng thật, hoàn toàn giả.

$$f_{\text{Nyquist}} = \frac{f_s}{2}, \quad f_{\text{giả}} = |f - kf_s|, \quad k = \text{round}(f/f_s)$$

- f_s : tần số lấy mẫu; f : tần số của dao động thật; $f_{\text{giả}}$: tần số nó giả dạng khi $f > f_{\text{Nyquist}}$.

Nói bằng lời. Chỉ thấy đúng dao động chậm hơn một nửa tần số lấy mẫu. Dao động nhanh hơn thì "gập" xuống: lấy tần số của nó trừ bội số gần nhất của tần số lấy mẫu. Ở ví dụ: sóng 1/4 mỗi bước, lấy mẫu 1/3 mỗi bước, Nyquist 1/6; $|1/4 - 1/3| = 1/12$, tức chu kỳ giả 12 bước.

Aliasing: thành phần 1,4 chu kỳ/giờ gập xuống thành 0,4 chu kỳ/giờ khi hạ mẫu mà không lọc trước



Cách đọc hình.

1. **Trục ngang:** ô trái là giờ 0 → 30; ô phải là tần số sau khi hạ mẫu (chu kỳ/giờ), 0 → 0,5.
2. **Trục dọc:** ô trái là giá trị; ô phải là mật độ phổ, thang log.
3. **Ký hiệu:** xám là tín hiệu mô phỏng 10 phút một mẫu (nhịp ngày + dao động 43 phút); cam là hạ mẫu thô (lấy mỗi 6 mẫu một); xanh là lọc thông thấp rồi mới hạ mẫu.
4. **Nhìn vào đâu:** ô phải, đỉnh cam ở vạch 0,4.
5. **Kết luận:** dao động 43 phút (1,4 chu kỳ/giờ) nhanh hơn Nyquist mới, nên gập thành một chu kỳ giả 2,5 giờ; lọc trước thì đỉnh đó biến mất.

Công suất ở chu kỳ 2,5 giờ sau khi hạ mẫu	Hạ mẫu thô	Lọc trước rồi hạ mẫu
Tín hiệu mô phỏng có dao động 43 phút	64,0	0,0

Độc bảng. Đỉnh 64,0 là sản phẩm của cách hạ mẫu, không phải của dữ liệu. Tính bằng công thức:
 $|1,4 - 1 \times 1| = 0,4$ chu kỳ/giờ.

Khi nào dùng, khi nào không. Trước mọi lần hạ mẫu, lọc thông thấp ở khoảng 0,8 lần Nyquist mới (`ha_mau(..., loc_truoc=True)`). `resample("h").mean()` đã là một bộ lọc trung bình, tốt hơn lấy một mẫu

(`.first()`), nhưng cắt không sắc; khi dao động nhanh mạnh thì vẫn phải lọc tử tế. Bộ lọc chống aliasing trong code chạy hai chiều, nên chỉ dùng để tiền xử lý cả chuỗi lịch sử, không dùng trong feature.

Tóm lại. Lấy mẫu với tần số f_s chỉ thấy đúng dao động chậm hơn $f_s/2$; dao động nhanh hơn giả dạng thành dao động chậm. Lọc thông thấp trước khi hạ mẫu.

Tự kiểm tra. Dữ liệu mỗi 10 phút có dao động chu kỳ 20 phút. Hạ mẫu thành mỗi 15 phút mà không lọc. Dao động đó giả dạng thành chu kỳ bao nhiêu phút?

Đáp án (tự làm trước rồi mới đọc)

Tính theo chu kỳ mỗi giờ: $f = 3$ (chu kỳ 20 phút), $f_s = 4$ (15 phút một mẫu), Nyquist = $2 < 3$ nên bị gập. $k = \text{round}(3/4) = 1$; $f_{\text{giả}} = |3 - 4| = 1$ chu kỳ/giờ, tức **60 phút**. Nhảm hay gập: nghĩ dao động nhanh sẽ "biến mất" khi hạ mẫu; nó không mất mà giả dạng thành dao động chậm.

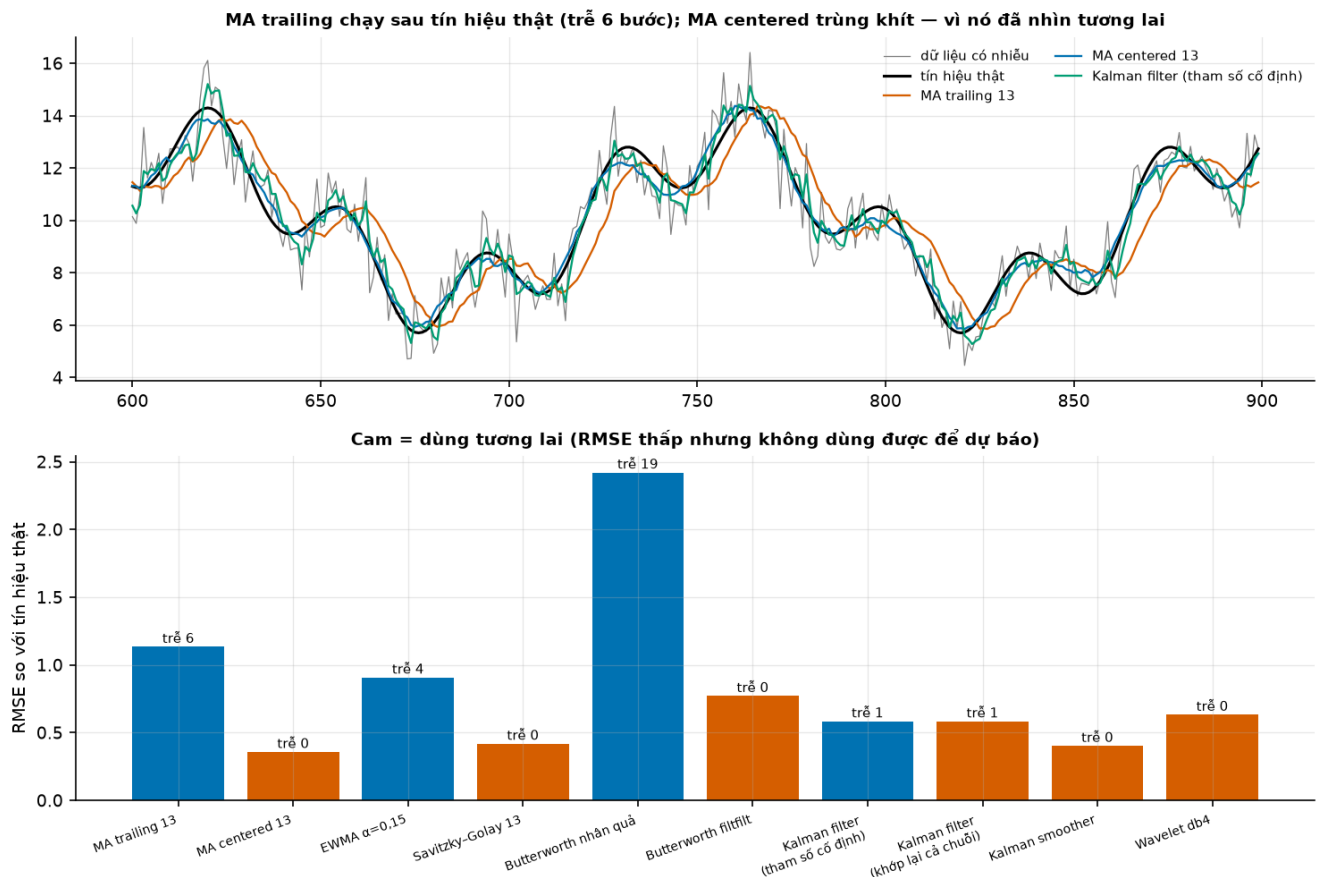
4.4 Sáu họ bộ lọc, và cái giá gọi là trễ

Vấn đề. Có nhiều bộ lọc; mỗi cái đổi độ trơn lấy trễ, hoặc lấy việc nhìn tương lai. Cần biết cái nào dùng được ở đâu.

Ví dụ số nhỏ — tự tính tay (EWMA). $z_t = \alpha y_t + (1 - \alpha)z_{t-1}$, $\alpha = 0,5$, chuỗi (10, 20, 10): $z_1 = 10$; $z_2 = 0,5 \times 20 + 0,5 \times 10 = 15$; $z_3 = 0,5 \times 10 + 0,5 \times 15 = 12,5$. Mỗi đầu ra chỉ cần đầu ra trước và số mới: nhân quả, nhớ rất ít. (pandas mặc định `adjust=True` tính khác ở vài điểm đầu; về sau hai cách như nhau.)

Họ bộ lọc	Làm gì	Dùng khi	KHÔNG dùng khi	Nhân quả
trung bình trượt trailing	trung bình k điểm gần nhất	feature dự báo, báo cáo vận hành	cần biết đúng lúc đỉnh xảy ra (trễ $(k - 1)/2$)	có
trung bình trượt centered	trung bình quanh điểm	mô tả, tách xu hướng	mọi feature dự báo	không
EWMA	cộng dồn có trọng số giảm dần về quá khứ	dữ liệu đang chảy về, cần trễ nhỏ	cần cắt đúng một dải tần số	có
Savitzky-Golay	khớp đa thức bậc thấp trên cửa sổ quanh điểm	giữ chiều cao đỉnh (cảm biến, phổ học)	feature dự báo, kể cả ở cuối chuỗi	không
Butterworth <code>sosfilt</code> / <code>sosfiltfilt</code>	cắt tần số theo ngưỡng; một chiều / xuôi rồi ngược	cần bỏ đúng dải tần số cao	bản hai chiều cho feature dự báo	có / không
Kalman filter / smoother	ước lượng tín hiệu ẩn từ mô hình (mức đổi dần + nhiễu)	có mô hình hợp; dữ liệu có lỗi	smoother cho feature dự báo	có / không

Đọc bảng. Cột cuối là cột quyết định. Bản hai chiều (`filtfilt`) "không trễ" vì chạy ngược thời gian; Savitzky-Golay ở cuối chuỗi vẫn khớp đa thức trên cửa sổ cuối, dùng cả điểm sau t (tài liệu scipy nói rõ). Hamilton (2018) chỉ ra với bộ lọc Hodrick-Prescott (HP, hay dùng trong kinh tế học để tách xu hướng): giá trị lọc ở **cuối chuỗi** khác hẳn giá trị ở giữa. Mà cuối chuỗi chính là nơi dự báo bắt đầu.



Cách đọc hình.

- Trục ngang:** ô trên là chỉ số thời gian 600 → 900 của tín hiệu mô phỏng; ô dưới là 10 cấu hình bộ lọc.
- Trục dọc:** ô trên là giá trị; ô dưới là RMSE so với tín hiệu thật (biết trước, vì là mô phỏng: nhịp 144 bước + nhịp 36 bước + nhiễu, seed 0).
- Ký hiệu:** ô trên, đen là tín hiệu thật, xám là dữ liệu có nhiễu, mỗi màu một bộ lọc; ô dưới, xanh là nhân quả, cam là nhìn tương lai; số trên cột là trễ (bước).
- Nhìn vào đâu:** ô trên, đường cam (trailing) so với đường đen; ô dưới, ba cột thấp nhất.
- Kết luận:** ba RMSE thấp nhất (centered, Kalman smoother, Savitzky-Golay) đều nhìn tương lai. Trong nhóm nhân quả, Kalman filter tốt nhất (0,584); Butterworth nhân quả tệ nhất (2,419) vì trễ 19 bước.

Đo trễ. Dịch đầu ra lùi k bước, tính tương quan với tín hiệu thật, chọn k cho tương quan lớn nhất (`trepha` , như tương quan chéo buổi 8). Trung bình trượt 13 điểm đo được trễ 6 bước, đúng bằng $(13 - 1)/2$. EWMA $\alpha = 0,15$ đo được trễ 4 (lý thuyết khoảng $(1 - \alpha)/\alpha \approx 5,7$), nhỏ hơn trung bình trượt cùng độ trơn.

pandas cho EWMA ba cách khai tham số: $\alpha = 2/(\text{span} + 1) = 1/(1 + \text{com})$. Luôn ghi rõ đã dùng tham số nào, nếu không người khác tái lập ra một bộ lọc khác.

Nâng cao — có thể bỏ qua lần đọc đầu. Wavelet tách chuỗi thành nhiều thang (`pywt.wavedec`), co các hệ số nhỏ về 0 bằng ngưỡng $\sigma\sqrt{2\ln n}$ (Donoho & Johnstone, 1994), với σ ước lượng bằng MAD của thang mịn nhất, rồi ghép lại. Ưu điểm: giữ được bước nhảy (dịch mức) mà trung bình trượt bôi nhòe. Nhược điểm: ghép lại dùng cả chuỗi, và σ tính trên cả chuỗi: không nhân quả.

Tóm lại. Độ trơn luôn phải trả bằng trễ, hoặc bằng việc nhìn tương lai. Khi xếp hạng bộ lọc cho dự báo, luôn có cột "nhân quả", và chỉ so các bộ lọc nhân quả với nhau.

Tự kiểm tra. EWMA với $\alpha = 0,2$, $z_{t-1} = 50$, số mới $y_t = 100$. Tính z_t . Nếu khai `span=9` thì α bằng bao nhiêu?

Đáp án (tự làm trước rồi mới đọc)

$z_t = 0,2 \times 100 + 0,8 \times 50 = 60$. `span = 9` $\rightarrow \alpha = 2 / (9 + 1) = 0,2$: cùng bộ lọc. Nhầm hay gặp: truyền `com=9` rồi tưởng là cùng bộ lọc; `com=9` cho $\alpha = 1/10 = 0,1$, trơn hơn và trễ hơn.

4.5 Bài kiểm nhân quả: đổi đuôi, xem đầu

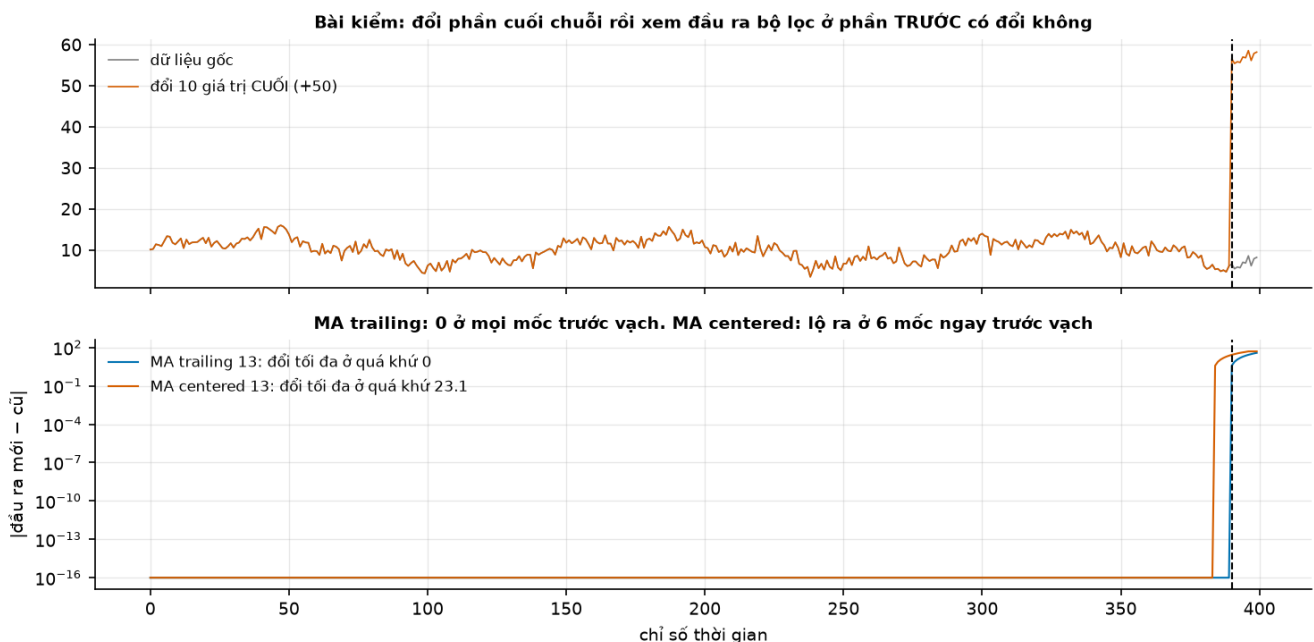
Vấn đề. Tài liệu thư viện không phải lúc nào cũng nói rõ; và một bộ lọc nhân quả vẫn rò rỉ nếu **tham số** của nó ước lượng trên cả chuỗi. Cần một phép thử chạy được trên mọi hàm.

Trực giác. Đúng như ví dụ mục 4.1: đổi vài giá trị **cuối** chuỗi, chạy lại bộ lọc, so đầu ra ở **mọi mốc trước** đó. Nhân quả thì phần trước không nhúc nhích.

```
goc = ham_loc(y)
y2 = y.copy(); y2[-10:] += 50.0          # chỉ đổi 10 điểm cuối
moi = ham_loc(y2)
lech = np.abs(moi[:-10] - goc[:-10])     # so ở MỌI mốc trước đó
nhan_qua = lech.max() <= max(1e-9, 1e-7 * np.max(np.abs(y)))
```

Ba chi tiết dễ làm sai:

- phải so **mọi** mốc trước điểm đổi: bỏ qua một đoạn cuối "cho chắc" là bỏ đúng chỗ lộ vi phạm;
- dung sai theo **thang** dữ liệu, vì phép tính số thực có sai số rất nhỏ;
- `np.nan_to_num` trước khi so, vì trung bình trượt sinh NaN ở đầu chuỗi.



Cách đọc hình.

1. **Trục ngang** (cả hai ô): chỉ số thời gian 0 $\rightarrow$ 400.
2. **Trục dọc**: ô trên là giá trị; ô dưới là $|\text{đầu ra mới} - \text{đầu ra cũ}|$, thang log.
3. **Ký hiệu**: vạch đen đứt là điểm bắt đầu đổi (390); ô dưới, xanh là trailing 13, cam là centered 13.
4. **Nhìn vào đầu**: ô dưới, phần bên trái vạch đen.

5. **Kết luận:** trailing bằng 0 ở mọi mốc trước vạch; centered đối ở 6 mốc ngay trước vạch (nửa cửa sổ 13).

10 cấu hình (tín hiệu mô phỏng)	Nhìn tương lai?	Quá khứ đối tối đa
trung bình trượt trailing 13	không	0,000
trung bình trượt centered 13	có	23,077
EWMA $\alpha = 0,15$	không	0,000
Savitzky-Golay 13	có	20,629
Butterworth nhân quả (<code>sosfilt</code>)	không	0,000
Butterworth <code>filtfilt</code>	có	23,615
Kalman filter, tham số ước lượng trên phần học rồi cố định	không	0,000
Kalman filter, tham số khớp lại trên cả chuỗi	có	1,134
Kalman smoother	có	17,898
wavelet db4	có	7,733

Độc bảng. Sáu cấu hình nhìn tương lai. Đáng nhớ nhất là hai dòng Kalman filter: cùng một bộ lọc nhân quả, chỉ khác chỗ lấy tham số, mà một cái rò rỉ. Với `filtfilt`, mốc đầu tiên bị đối ở vị trí 126 trên 400: nó làm bắn ngược 274 bước về quá khứ.

Khi nào dùng, khi nào không. Chạy bài kiểm này cho **mọi** hàm sinh feature trước khi dùng, kể cả hàm "chắc chắn nhân quả". Không coi nó là đủ: bài kiểm chỉ bắt rò rỉ qua đuôi chuỗi; rò rỉ qua cách chia tập (buổi 13) cần kiểm khác.

Tóm lại. Đối đuôi chuỗi, so mọi mốc trước đó: đối là nhìn tương lai. Tham số ước lượng trên cả chuỗi cũng là nhìn tương lai; ước lượng trên phần học rồi cố định.

Tự kiểm tra. Một bộ lọc cho "quá khứ đối tối đa" = 0,0000001 trên chuỗi có giá trị lớn nhất 1.000. Nhân quả không?

Đáp án (tự làm trước rồi mới đọc)

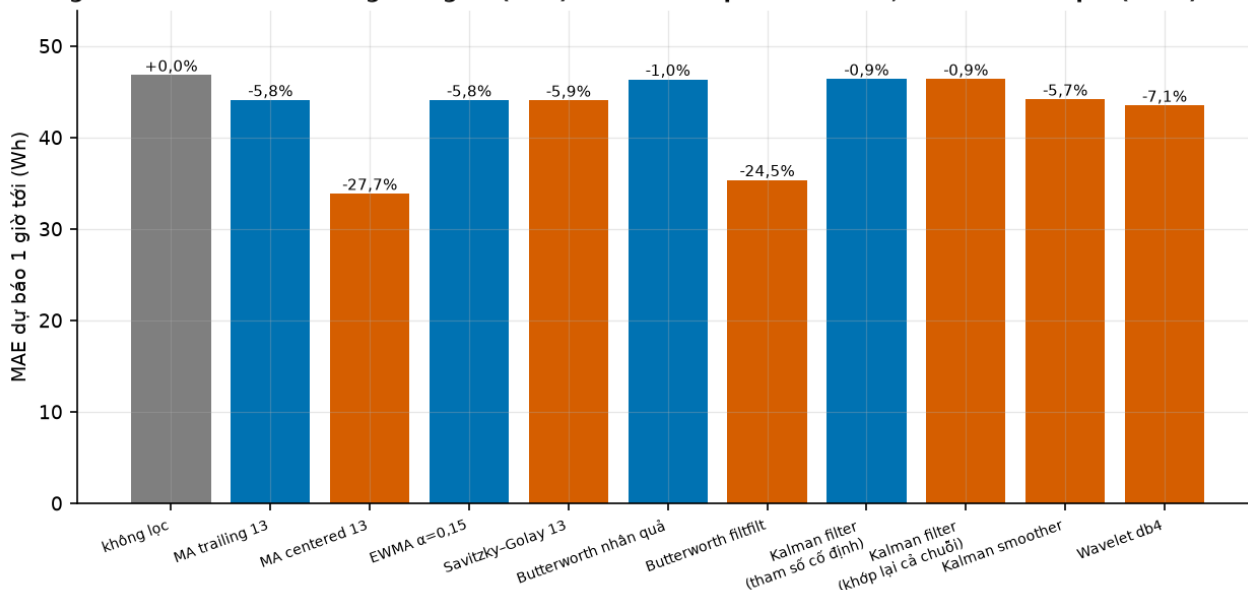
Ngưỡng là $\max(10^{-9}, 10^{-7} \times 1.000) = 10^{-4}$; 10^{-7} nhỏ hơn nhiều, nên coi là **nhân quả**: đó là sai số tính toán, không phải nhìn tương lai. Nhầm hay gặp: đặt ngưỡng tuyệt đối 0 rồi báo động giả cho mọi bộ lọc tính bằng số thực.

4.6 Cái giá của rò rỉ, và làm trơn mục tiêu

Vấn đề. Feature nhìn tương lai làm sai số đẹp bao nhiêu? Và nếu làm trơn chính **mục tiêu** thì sao?

Cách đo. Dự báo điện thiết bị một giờ tới (6 bước) bằng hồi quy tuyến tính trên hai biến: đầu ra bộ lọc và giá trị hiện tại (như hồi quy đơn, thêm một biến). Học trên phần đầu (70%), chấm MAE trên phần sau, **trên chuỗi gốc**.

Cái giá của rõ rĩ: feature dùng tương lai (cam) cho MAE thấp hơn 24-28%, feature nhân quả (xanh) chỉ 1-6%



Cách đọc hình.

- Trục ngang:** không lọc và 10 cấu hình bộ lọc.
- Trục dọc:** MAE dự báo 1 giờ tới (Wh).
- Ký hiệu:** xám là không lọc; xanh là nhân quả; cam là nhìn tương lai; số trên cột là phần trăm đổi so với không lọc.
- Nhìn vào đâu:** hai cột cam thấp nhất; các cột xanh.
- Kết luận:** trung bình trượt centered và `filtfilt` trông như feature tuyệt vời (giảm hơn một phần tư sai số); mọi feature nhân quả chỉ giúp vài phần trăm.

Không lọc: MAE 46,84 Wh. Con số $-27,7\%$ là "phần thưởng" mà mô hình sẽ **mất sạch** khi dùng thật, vì lúc đó không có số của giờ sau. Nguy hiểm hơn là Savitzky-Golay và wavelet: cũng rõ rĩ, nhưng chỉ giảm sai số một chút, trông hợp lý. Chỉ bài kiểm tự động bắt được chúng.

Làm trơn mục tiêu. Cùng mô hình, chấm trên hai mục tiêu:

Chấm trên	MAE (Wh)
điện thật	46,84
điện đã làm trơn bằng trung bình trượt centered 13	35,78

Đọc bảng. Sai số giảm 23,6% mà mô hình không hề tốt hơn: nó chỉ được chấm trên một đại lượng dễ hơn và không có thật (không ai trả tiền điện "đã làm trơn"). Hình [hình/muc-tieu-lam-tron.png](#) (ô bước 5) vẽ hai mục tiêu chống lên nhau: đường làm trơn cắt mất các đỉnh nhọn, đúng những chỗ mô hình sai nhiều nhất.

Năm câu hỏi chọn bộ lọc, theo thứ tự.

- Đầu ra dùng làm gì? Mô tả thì bộ lọc nào cũng được; feature hay mục tiêu thì chỉ nhân quả.
- Phổ nói gì? Tần số cao gần như không có năng lượng thì lọc chỉ thêm trễ.
- Chịu trễ bao nhiêu? Bộ lọc trễ 19 bước cho dự báo 6 bước tới gần như vô dụng (chỉ giúp 1,0%).
- Chuỗi có bước nhảy không? Có thì tìm điểm gãy trước (buổi 11) rồi lọc từng đoạn.
- Tham số lấy từ đâu? Chỉ từ phần học, rồi cố định và ghi lại.

Tóm lại. Feature nhìn tương lai làm sai số đẹp giả tạo, có khi chỉ vài phần trăm nên không ai nghi. Được làm trơn feature bằng bộ lọc nhân quả; không bao giờ làm trơn mục tiêu dùng để chấm.

Tự kiểm tra. Sếp muốn dự báo "điện trung bình 3 giờ tới". Có được làm trơn mục tiêu không?

Đáp án (*tự làm trước rồi mới đọc*)

Được, nếu **định nghĩa lại mục tiêu** thành trung bình 3 giờ tới (cộng dồn các giờ tương lai) và ghi rõ trong báo cáo; đó là một đại lượng có thật mà sếp cần. Không được lấy trung bình trượt centered của điện rồi gọi là "điện": nó trộn cả giờ trước và giờ sau. Nhầm hay gặp: làm trơn mục tiêu "để mô hình dễ học" rồi báo sai số trên mục tiêu đã làm trơn.

5. Lab từng bước

Lệnh gõ trong terminal ở thư mục `lab/`. Code các bước nằm sẵn trong `code/lab.ipynb` (mở bằng `python lab.py notebook` hoặc VS Code), chạy từng ô từ trên xuống. Bạn sửa `code/khu_nhieu.py`; notebook tự nạp lại bản mới.

Bước 1 — Nhìn phổ trước khi lọc

Mục đích: đọc phổ hai cảm biến (mục 4.2) và thử lại ví dụ tay mục 4.1.

```
python lab.py up          # một lần: môi trường + dữ liệu Appliances Energy, kiểm sha256
python lab.py check       # 6/8 test đỏ
python lab.py notebook    # chạy ô bước 1
```

Độc kết quả: chu kỳ mạnh nhất 24,38 giờ ở cả hai; phần năng lượng ở chu kỳ dưới 1 giờ: 0,178 và 0,0. Ví dụ 5 giờ in hai kiểu trung bình trượt như bảng mục 4.1.

Bước 2 — Aliasing

Mục đích: sửa `ha_mau` để cờ `loc_truoc` có tác dụng (mục 4.3): lọc Butterworth bậc 8 ở 0,8 lần Nyquist mới rồi mới lấy mẫu. Chạy lại ô bước 2.

Độc kết quả: công suất ở chu kỳ 2,5 giờ như bảng mục 4.3. Ô cũng thử dao động 1,5 chu kỳ/giờ: nó gặp đúng vào Nyquist mới, mọi mẫu lấy được đều bằng 0, và phổ không thấy gì: một cái bẫy khi tự dựng ví dụ aliasing.

Bước 3 — Bài kiểm nhân quả

Mục đích: sửa `kiem_nhan_qua` để so **mọi** mốc trước điểm đổi (mục 4.5). Chạy lại ô bước 3.

Độc kết quả: bảng 10 cấu hình có cột "dùng tương lai?" đúng như bảng mục 4.5: 4 nhân quả, 6 nhìn tương lai.

Bước 4 — Giá của rò rỉ

Mục đích: sửa `danh_gia_feature` để không còn mặc định bộ lọc centered: tham số `ham_loc` bắt buộc truyền vào (mục 4.6). Chạy lại ô bước 4.

Độc kết quả: bảng MAE như hình mục 4.6. Viết ba câu cho sắp: nên dùng feature nào, vì sao không dùng feature "tốt nhất".

Bước 5 — Chấm đúng mục tiêu

Mục đích: sửa `cham_tren_muc_tieu_lam_tron` để trả **cả hai** MAE: `mae_muc_tieu_goc` và `mae_muc_tieu_lam_tron`. Chạy lại ô bước 5, rồi:

```
python lab.py check # 8/8 xanh
```

Độc kết quả: 46,84 và 35,78. Xanh 8/8 là xong; `test_kiem_nhan_qua_xet_moi_moc_truoc_diem_doi` còn đỏ thì `kiem_nhan_qua` vẫn bỏ qua đoạn cuối.

6. Lỗi thường gặp & cách chẩn đoán

Triệu chứng	Nguyên nhân	Chẩn đoán	Sửa
Backtest đẹp bất thường sau khi thêm feature làm tròn	bộ lọc nhìn tương lai	bài kiểm đổi đuôi	<code>trailing</code> , <code>EWMA</code> , <code>sosfilt</code> , Kalman filter
" <code>filtfilt</code> không trở nên tốt hơn"	không trở vì chạy ngược thời gian	mốc đầu tiên bị đổi	<code>filtfilt</code> chỉ để mô tả
Savitzky-Golay "chắc nhàn quả ở cuối"	ở cuối vẫn khớp đa thức trên cửa sổ cuối	bài kiểm đổi đuôi	không dùng làm feature
Chu kỳ lạ xuất hiện sau <code>resample</code>	aliasing	so phổ trước/sau; tính $ f - kf_s $	lọc thông thấp trước khi hạ mẫu
Bộ lọc nhân quả mà vẫn rò rỉ	tham số ước lượng trên cả chuỗi	bài kiểm đổi đuôi vẫn đỏ	ước lượng trên phần học rồi cố định
MAE giảm mạnh mà dự báo trông như cũ	chấm trên mục tiêu đã làm tròn	in MAE trên mục tiêu gốc	luôn chấm trên chuỗi gốc
Khử nhiễu làm mất dịch mức	bộ lọc bôi nhoe bước nhảy	so quanh mốc gãy	tìm điểm gãy trước (buổi 11); wavelet
Kalman "tốt hơn hẳn"	dùng smoother	so <code>filtered_state</code> với <code>smoothed_state</code>	filter cho dự báo
Bài kiểm báo động giả	dung sai tuyệt đối quá chặt	in "quá khứ đổi tối đa"	dung sai theo thang dữ liệu
EWMA tái lập không khớp	lỗi <code>alpha</code> , <code>span</code> , <code>com</code>	in tham số	ghi rõ tham số đã dùng

7. Bài tập về nhà

- Quét α .** Với EWMA, quét $\alpha = 0,05; 0,1; \dots; 0,9$, vẽ MAE dự báo theo α . Có điểm tốt nhất ở giữa không? Giải thích bằng đánh đổi nhiễu-trễ.
- Bộ lọc HP.** Chạy `statsmodels.tsa.filters.hp_filter.hpfilter` trên `T2`, áp bài kiểm nhân quả. Rồi cắt 100 điểm cuối, chạy lại, so thành phần xu hướng ở 100 điểm **trước** đó: đúng điều Hamilton (2018) mô tả.
- Wavelet với bước nhảy.** Thêm một dịch mức +100 vào tín hiệu mô phỏng. So RMSE của wavelet db4 và trung bình trượt trailing 13 quanh mốc nhảy (± 20 điểm) và ở phần còn lại.

8. Tiêu chí "Xong khi"

- [] `python lab.py check` xanh 8/8.
- [] Tính tay được trung bình trượt 3 điểm hai kiểu và chỉ ra kiểu nào nhìn tương lai.
- [] Nộp bảng 10 bộ lọc có cột "dùng tương lai?" do bài kiểm tự động điền.
- [] Nêu con số aliasing: công suất ở chu kỳ 2,5 giờ là 64,0 (hạ mẫu thô) so với 0,0 (lọc trước), và tính được chu kỳ giả bằng công thức.
- [] Giải thích vì sao hai Kalman filter (tham số cố định / khớp lại trên cả chuỗi) cho RMSE gần như nhau mà chỉ một cái nhân quả.
- [] Viết một đoạn ngắn: vì sao MAE 35,78 trên mục tiêu đã làm tròn **không** được đưa vào báo cáo.

9. Đọc thêm

- `scipy.signal` — `welch`, `sosfilt`, `sosfiltfilt`, `savgol_filter`, `butter`: <https://docs.scipy.org/doc/scipy/reference/signal.html>
- Schaedler, J. *Circles, Sines and Signals* — bánh xe quay ngược: <https://jackschaedler.github.io/circles-sines-signals/sampling4.html>
- Hamilton, J.D. (2018). Why You Should Never Use the Hodrick-Prescott Filter. *REStat* 100(5), 831–843.
- Moura, A. (2024). A comment on Hamilton (2018). *Journal of Comments and Replications in Economics* 3.
- Donoho, D.L. & Johnstone, I.M. (1994). Ideal spatial adaptation by wavelet shrinkage. *Biometrika* 81(3), 425–455.
- Savitzky, A. & Golay, M.J.E. (1964). *Analytical Chemistry* 36(8), 1627–1639.
- Welch, P. (1967). The use of FFT for the estimation of power spectra. *IEEE Trans. Audio Electroacoust.* 15(2), 70–73.
- `statsmodels` `UnobservedComponents` — `filtered_state`, `smoothed_state`: <https://www.statsmodels.org/stable/statespace.html>
- PyWavelets — thresholding: <https://pywavelets.readthedocs.io/>
- Candanedo, L. (2017). Appliances Energy Prediction. UCI ML Repository. <https://doi.org/10.24432/C5VC8G>